

Report

Loss Function

The total loss is defined as:

$$\text{Total Loss} = \text{CrossEntropy Loss} + \lambda \times \text{Sparsity Loss}$$

Where:

- CrossEntropy Loss ensures classification performance
- Sparsity Loss is the average of all gate values

The sparsity loss encourages the network to deactivate unnecessary connections.

Why L1 Encourages Sparsity

The L1 penalty pushes parameters toward zero. Since gate values lie between 0 and 1, minimizing their sum encourages many gates to approach zero. This effectively removes connections, and makes the model sparse.

Results

Lambda	Test Accuracy	Sparsity (%)
0.001	0.4979	1.53
0.01	0.4945	2.05
0.1	0.4999	20.47

Observations

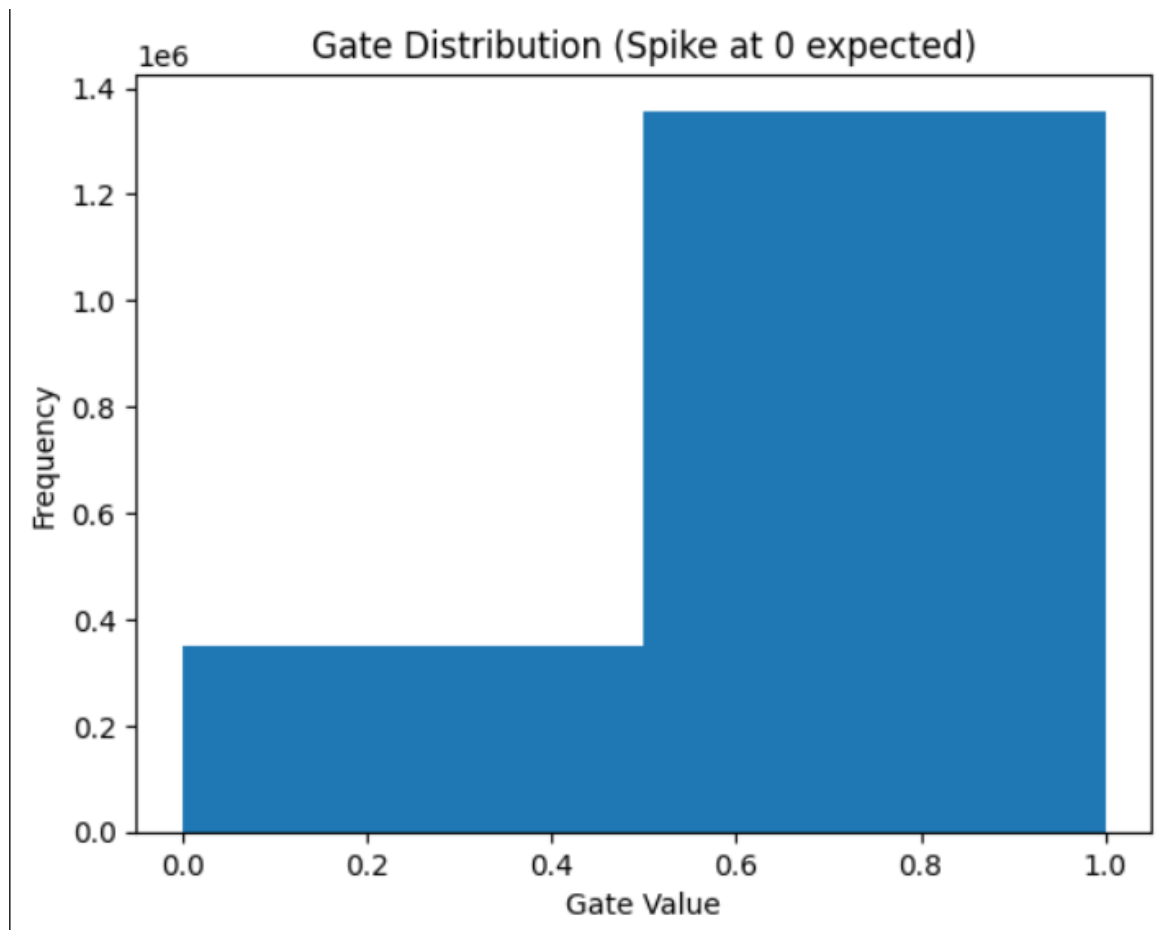
1. Increasing λ increases sparsity.
2. Higher sparsity reduces model accuracy.
3. There is a clear trade-off between model efficiency and performance.

Gate Distribution

The histogram of gate values shows:

- A large spike at 0: pruned connections

- A smaller spike at 1: important connections



Code

```
import torch

import torch.nn as nn

import torch.nn.functional as F

import torch.optim as optim

from torchvision import datasets, transforms

import matplotlib.pyplot as plt


device = torch.device("cuda" if torch.cuda.is_available() else "cpu")


class PrunableLinear(nn.Module):

    def __init__(self, in_features, out_features):

        super().__init__()
```

```
self.weight = nn.Parameter(torch.randn(out_features, in_features) * 0.01)
```

```
self.bias = nn.Parameter(torch.zeros(out_features))
```

```
self.gate_scores = nn.Parameter(torch.randn(out_features, in_features))
```

```
def forward(self, x):
```

```
    gates = torch.sigmoid(self.gate_scores)
```

```
    hard_gates = (gates > 0.1).float()
```

```
    gates = gates + (hard_gates - gates).detach()
```

```
    pruned_weights = self.weight * gates
```

```
    return F.linear(x, pruned_weights, self.bias)
```

```
class Net(nn.Module):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.fc1 = PrunableLinear(32*32*3, 512)
```

```
        self.fc2 = PrunableLinear(512, 256)
```

```
        self.fc3 = PrunableLinear(256, 10)
```

```
    def forward(self, x):
```

```
        x = x.view(x.size(0), -1)
```

```
        x = F.relu(self.fc1(x))
```

```
        x = F.relu(self.fc2(x))
```

```
        x = self.fc3(x)
```

```
        return x
```

```
def sparsity_loss(model):
```

```
    loss = 0
```

```
total = 0
```

```
for module in model.modules():
```

```
    if isinstance(module, PrunableLinear):
```

```
        gates = torch.sigmoid(module.gate_scores)
```

```
        loss += gates.sum()
```

```
        total += gates.numel()
```

```
return loss / total
```

```
def train(model, loader, optimizer, lambda_val):
```

```
    model.train()
```

```
    total_loss = 0
```

```
    for data, target in loader:
```

```
        data, target = data.to(device), target.to(device)
```

```
        optimizer.zero_grad()
```

```
        output = model(data)
```

```
        ce_loss = F.cross_entropy(output, target)
```

```
        sp_loss = sparsity_loss(model)
```

```
        loss = ce_loss + lambda_val * sp_loss
```

```
        loss.backward()
```

```
        optimizer.step()
```

```
        total_loss += loss.item()
```

```
    return total_loss / len(loader)
```

```
def test(model, loader):
```

```
model.eval()
```

```
correct = 0
```

```
with torch.no_grad():
```

```
    for data, target in loader:
```

```
        data, target = data.to(device), target.to(device)
```

```
        output = model(data)
```

```
        pred = output.argmax(dim=1)
```

```
        correct += pred.eq(target).sum().item()
```

```
return correct / len(loader.dataset)
```

```
def calculate_sparsity(model, threshold=0.1):
```

```
    total = 0
```

```
    zero = 0
```

```
    for module in model.modules():
```

```
        if isinstance(module, PrunableLinear):
```

```
            gates = torch.sigmoid(module.gate_scores)
```

```
            total += gates.numel()
```

```
            zero += (gates < threshold).sum().item()
```

```
    return 100 * zero / total
```

```
transform = transforms.Compose([
```

```
    transforms.ToTensor()
```

```
])
```

```
train_data = datasets.CIFAR10('./data', train=True, download=True, transform=transform)
```

```
test_data = datasets.CIFAR10('./data', train=False, download=True, transform=transform)
```

```
train_loader = torch.utils.data.DataLoader(train_data, batch_size=128, shuffle=True)
```

```
test_loader = torch.utils.data.DataLoader(test_data, batch_size=128)
```

```
lambdas = [0.001, 0.01, 0.1]
```

```
results = []
```

```
best_model = None
```

```
best_acc = 0
```

```
for lam in lambdas:
```

```
    print(f"\nTraining with lambda = {lam}")
```

```
    model = Net().to(device)
```

```
    optimizer = optim.Adam(model.parameters(), lr=1e-3)
```

```
    for epoch in range(10):
```

```
        loss = train(model, train_loader, optimizer, lam)
```

```
        print(f"Epoch {epoch+1}, Loss: {loss:.4f}")
```

```
    acc = test(model, test_loader)
```

```
    sparsity = calculate_sparsity(model)
```

```
    print(f"Accuracy: {acc:.4f}, Sparsity: {sparsity:.2f}%")
```

```
    results.append((lam, acc, sparsity))
```

```
    if acc > best_acc:
```

```
        best_acc = acc
```

```
        best_model = model
```

```
all_gates = []
```

```
for module in best_model.modules():
```

```
if isinstance(module, PrunableLinear):  
    gates = torch.sigmoid(module.gate_scores)  
    hard_gates = (gates > 0.1).float()  
    all_gates.extend(hard_gates.cpu().detach().numpy().flatten())  
  
plt.hist(all_gates, bins=2)  
plt.title("Gate Distribution (Spike at 0 expected)")  
plt.xlabel("Gate Value")  
plt.ylabel("Frequency")  
plt.show()  
  
print("\nFinal Results:")  
for lam, acc, sp in results:  
    print(f"Lambda: {lam}, Accuracy: {acc:.4f}, Sparsity: {sp:.2f}%")
```